

Explanation- Lab Programs.pptx (1).pptx

Exported: May 28, 2026

Summary

This instructional material demonstrates how to use the **MapReduce framework** to process and analyze large datasets, specifically focusing on **NCDC weather records** and relational data joins. The text outlines the technical workflow for extracting specific fields—such as **year, month, and temperature**—from raw data strings to calculate statistical insights like **annual averages and monthly mean maximums**. Beyond weather analysis, it explores more advanced data manipulation techniques, including **sorting word frequencies** and performing a **MapReduce join** to merge disparate files using a shared key. Ultimately, the source serves as a practical guide for building **Mappers, Reducers, and Drivers** to transform disorganized raw input into structured, meaningful summaries.

Keywords: MapReduce Program Examples, NCDC Weather Data, Average Temperature Calculation, Dataset Joining Methods, Hadoop Data Processing

Content

Question: From the following link extract the weather data: <https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all> Create a Map Reduce program to: a) Find average temperature for each year from NCDC data set. b) Find the mean max temperature for every month.

Dataset Description: If we consider the details mentioned in the file, each file has entries that look like this: 00290290709999991905010106004+64333+023450FM2+000599999V0202301N008219999999

we consider the highlighted fields, the first one (029070) is the USAF weather station identifier. The next one (19050101) represents the observation date. The third highlighted one (0139) represents the air temperature in Celsius times ten. So the reading of 0139 equates to 13.9 degrees Celsius. The next highlighted and italic item indicates a reading quality code. The input to our map phase is the raw NCDC data. To visualize the way the map works, consider the following sample lines of

input data (some unused columns have been dropped to fit the page, indicated by ellipses):0067011990999991950051507004...9999999N9+00001+9999999999...
 0043011990999991950051512004...9999999N9+00221+9999999999...
 0043011990999991950051518004...9999999N9-00111+9999999999...
 0043012650999991949032412004...0500001N9+01111+9999999999...
 0043012650999991949032418004...0500001N9+00781+9999999999...These

lines are presented to the map function as the key-value pairs:The map function merely extracts the year and the air temperature(indicated in bold text), and emits them as its output (the temperature values have been interpreted as integers): (1950, 0)(1950, 22)(1950, -11)(1949, 111)(1949, 78)our reduce function sees the following input:(1949, [111, 78])(1950, [0, 22, -11])All the reduce function has to do now is iterate through the list and pick up the maximum reading:(1949, 111)(1950, 22)a) Find average temperature for each year from NCDC data set.

```

AverageDriver:package temp;import org.apache.hadoop.fs.Path;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.Text;import
org.apache.hadoop.mapreduce.Job;import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;public class
AverageDriver {public static void main(String[] args) throws Exception {if
(args.length != 2) {System.err.println("Please Enter the input and output
parameters");System.exit(-1);}Job job = new
Job();job.setJarByClass(AverageDriver.class);job.setJobName("Max
temperature");FileInputFormat.addInputPath(job, new
Path(args[0]));FileOutputFormat.setOutputPath(job, new
Path(args[1]));job.setMapperClass(AverageMapper.class);job.setReducerClass(AverageReducer.cl
0 : 1);}}AverageMapper:package temp;import java.io.IOException;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.LongWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Mapper;public
class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{public static final int MISSING = 9999;public void map(LongWritable key, Text
value, Mapper<LongWritable, Text, Text,IntWritable>.Context context) throws
IOException, InterruptedException {int temperature;String line =
value.toString();String year = line.substring(15, 19);if (line.charAt(87) == '+')
{temperature = Integer.parseInt(line.substring(88, 92));} else {temperature =
Integer.parseInt(line.substring(87, 92));}String quality = line.substring(92, 93);if
(temperature != 9999 && quality.matches("[01459"]))context.write(new Text(year),
new IntWritable(temperature));}}AverageReducer:package temp;import
java.io.IOException;import org.apache.hadoop.io.IntWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Reducer;public
class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {public
void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable,Text,

```

```

IntWritable>.Context context) throws IOException, InterruptedException {int
max_temp = 0;int count = 0;for (IntWritable value : values) {max_temp +=
value.get();count++;}context.write(key, new IntWritable(max_temp / count));}}b)
find the mean max temperature for every monthMeanMaxDriver.classpackage
meanmax;import org.apache.hadoop.fs.Path;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.Text;import
org.apache.hadoop.mapreduce.Job;import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;public class
MeanMaxDriver {public static void main(String[] args) throws Exception {if
(args.length != 2) {System.err.println("Please Enter the input and output
parameters");System.exit(-1);}}Job job = new
Job();job.setJarByClass(MeanMaxDriver.class);job.setJobName("Max
temperature");FileInputFormat.addInputPath(job, new
Path(args[0]));FileOutputFormat.setOutputPath(job, new
Path(args[1]));job.setMapperClass(MeanMaxMapper.class);job.setReducerClass(MeanMaxReducer
0 : 1);}}MeanMaxMapper.classpackage meanmax;import java.io.IOException;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.LongWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Mapper;public
class MeanMaxMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{public static final int MISSING = 9999;public void map(LongWritable key, Text
value, Mapper<LongWritable, Text, Text,IntWritable>.Context context) throws
IOException, InterruptedException {int temperature;String line =
value.toString();String month = line.substring(19, 21);if (line.charAt(87) == '+')
{temperature = Integer.parseInt(line.substring(88, 92));} else {temperature =
Integer.parseInt(line.substring(87, 92));}String quality = line.substring(92, 93);if
(temperature != 9999 && quality.matches("[01459"]))context.write(new
Text(month), new IntWritable(temperature));}}MeanMaxReducer.classpackage
meanmax;import java.io.IOException;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.Text;import
org.apache.hadoop.mapreduce.Reducer;public class MeanMaxReducer extends
Reducer<Text, IntWritable, Text, IntWritable> {public void reduce(Text key,
Iterable<IntWritable> values, Reducer<Text, IntWritable,Text, IntWritable>.Context
context) throws IOException, InterruptedException {int max_temp = 0;int total_temp
= 0;int count = 0;int days = 0;for (IntWritable value : values) {int temp =
value.get();if (temp > max_temp)max_temp = temp;count++;if (count == 3)
{total_temp += max_temp;max_temp = 0;count = 0;days++;}}context.write(key,
new IntWritable(total_temp / days));}}For a given Text file, Create a Map Reduce
program to sort the content in an alphabetic order listing only top 10 maximum
occurrences of wordsHow to Join two DataSets: MapReduce ExampleThere are two
Sets of Data in two Different Files (shown below). The Key Dept_ID is common in

```

both files. The goal is to use MapReduce Join to combine these files

```
HADOOP_HOME/bin/hdfs dfs -copyFromLocal DeptStrength.txt DeptName.txt HADOOP_HOME/bin/hadoop jar MapReduceJoin.jar MapReduceJoin/JoinDriver/DeptStrength.txt / DeptName.txt /output_mapreducejoin$HADOOP_HOME/bin/hdfs dfs -cat /output_mapreducejoin/part-00000
```